

gretl working papers

Bayesian Averaging of Classical Estimates (BACE) for **gretl**

Marcin Błażejowski

WSB University in Torun

marcin.blazejowski@wsb.torun.pl

Jacek Kwiatkowski

Nicolaus Copernicus University in Toruń

jkwiat@umk.pl

working paper #6

August 25, 2018

Abstract

This paper presents a software package that implements Bayesian Averaging of Classical Estimates **BACE** ver. 1.3 for the **gretl** (the GNU regression, econometrics and time-series library).

Project's main page:

https://sourceforge.net/projects/bace-for-gretl

Warning

This PDF is outdated and needs to be updated.

Contents

1 Introduction **2**

2 Bayesian Averaging of Classical Estimates **3**

3 Usage of the BACE package **5**

3.1 BACE main function 5

3.2 The GUI way 6

3.3 The script way 8

3.3.1 Script 8

3.4 Output 10

Changelog

1.3 Massive changes and improvements:

- (to doc) New BIC types.
- (to doc) Correct model priors for truncated distributions.
- New MC3 strategy: taken directly from BMA package.
- New jointness measures.
- New `BACE()` bundle-based interface (default).
- Fix bug in `BACE_GUI()` in constant type selection.
- Fixed bug when „can be dropped" option is used.
- (to doc) New model types.
- (to doc) Updated docs.
- (to doc) Adding possibility for centering or standardizing variables.
- (do doc) New BIC versions.
- (do doc) Graphs added.
- Clean up the code.

1.2 Several changes:

- Some typos in output.
- Getting rid of calls to deprecated functions.
- New jointness measures.

1.1 Several changes:

- Documentation in PDF.
- Fix of several issues and bugs for model with only one explanatory variable and without constant.
- Change the option of the constant type from „random" to „can be dropped".

1.0 Initial version.

1 Introduction

In article Xavier Sala-i-Martin and Gernot Doppelhofer and Ronald I. Miller (2004) authors suggest a technique for averaging the parameter estimates – Bayesian Averaging of Classical Estimates (BACE), which enables to measure the importance of particular potential regressors. This approach is alternative to the familiar and earlier-applied Bayesian Model Averaging (BMA) technique, from which it differs mainly because of its less restrictive *prior* assumptions. The authors showed that

BACE weighting method may be understood as a limiting case of BMA approach when the *prior* information is dominated by the data. The averaging of the parameter estimates is performed across all possible combinations of linear regression models obtained by means of Ordinary Least Squares (OLS) with classical assumptions.

The BACE approach has several interesting features in relation to the techniques such as BMA or robustness-checking methods (reference). First of all, it is not necessary to make assumptions concerning *prior* distribution for parameters but only one simple assumption, on the expected model size.

An example of the application of the BACE technique for forecasting macroeconomic indicators can be found, among others, in the articles van Dijk (2004); Albis and Mapa (2014); Białowolski et al. (2014).

2 Bayesian Averaging of Classical Estimates

Consider simple regression model M_j:

$$y = X_j\beta_j + \epsilon \tag{1}$$

where y is $(Tx1)$ vector of observations, X_j is (Txk_j) matrix containing the set of independent variables and constant, β_j is a (k_jx1) vector of unknown parameters, ϵ is a $(Nx1)$ vector of errors which are assumed to be normally distributed i.e. $\epsilon \sim N(0_T, \sigma^2 I_T)$, $N(\mu, \Sigma)$ denotes a normal distribution with location μ and covariance Σ.

By applying Ordinary Least Squares (OLS) estimates (see Xavier Sala-i-Martin and Gernot Doppelhofer and Ronald I. Miller (2004)) we can calculate approximation of posterior probability of model M_j i.e. $\Pr(M_j|y)$ using the following formula: (add: Kass and Raftery (1995))

$$\Pr(M_j|y) \approx \frac{\Pr(M_j)T^{-k_j/2}SSE_j^{-T/2}}{\sum_{i=1}^{2^K}\Pr(M_i)T^{-k_i/2}SSE_i^{-T/2}}, \tag{2}$$

where SSE_j and SSE_i are the OLS sum of squared errors, 2^K denotes the total number of potential combinations of the K independent variables, k_j and k_i are the number of regression parameters β_j and β_i. Prior probabilities $\Pr(M_j)$ and $\Pr(M_i)$ of models M_j and M_i are binomially distributed, namely:

$$\Pr(M_j) = \theta^{k_i}(1-\theta)^{K-k_i}, \theta \in [0,1]. \tag{3}$$

Note that the Binomial distribution implies that we only need to specify a prior expected model size $\mathsf{E}(\Xi) = K\theta$ where $\mathsf{E}(\Xi) \in (0, K]$. If we define the value of $\mathsf{E}(\Xi)$ our BACE package will automatically produce a value of prior inclusion probability for all competitive models. In case of $\theta = 0.5$ prior expected model size is equal to the average of number of potential regressors, model prior distribution is uniform $\left(\Pr(M_i) = 2^{-K}\right)$ and reflects a lack of previous knowledge about models.

Add Beta-Binomial distribution (including truncated Atwood (1980); Tripathi et al. (1994))

Using BACE we can also easily evaluate the mean and variance of the posterior distribution of regression parameters β across the whole model space (see Leamer (1978)):

$$E(\beta|y) \approx \sum_{i=1}^{2^K} \Pr(M_i|y) E(\beta_i|y, M_i), \tag{4}$$

$$Var(\beta|y) \approx \sum_{i=1}^{2^K} \Pr(M_i|y) Var(\beta_i|y, M_i) + \sum_{i=1}^{2^K} \Pr(M_i|y) \left(E(\beta_i|y, M_i) - E(\beta|y)\right)^2, \tag{5}$$

where $E(\beta_i|y, M_i)$ and $Var(\beta_i|y, M_i)$ are the OLS estimates of β_i from model M_i.

Another useful characteristic popular in BACE approach is a Posterior Inclusion Probability (PIP). It is defined as the posterior probability that the variable x_i is relevant in explaining the dependent variable (see Leamer (1978); Mitchell and Beauchamp (1988)). In our case the PIP is calculated (using Monte Carlo integration) as the sum of the posterior model probabilities for all of the models including specific variable:

$$\Pr(\beta_i \neq 0|y) = \sum_{i=1}^{2^K} \Pr(M_i|y) \tag{6}$$

In many cases, despite of the interest of posterior characteristics it is valuable to predict future values of the dependent variable. In BACE approach we can carry out predictive inference across the whole model space. Given the matrix X_j^* containing explanatory variables for each of the out-of-sample data points we can predict inference on unobserved values of dependent variable as a weighted average of the individual model forecasts.

Doppelhofer and Weeks (2009)(DW2)

$$J_{DW2}(i,j) = \ln\left(\frac{p(i \cap j|y)p(\bar{i} \cap \bar{j}|y)}{p(i \cap \bar{j}|y)p(\bar{i} \cap j|y)}\right) \tag{7}$$

Ley and Steel (2007)(LS2)

$$J_{LS1}(i,j) = \ln\left(\frac{P(i \cap j|y)}{p(i|y) + p(j|y) - 2p(i \cap j|y)}\right) \tag{8}$$

Strachan Measure Strachan (2009)

$$J_{St}(i,j) = \; p(i|y)p(j|y) \ln\left(\frac{P(i \cap j|y)}{p(i \cap \bar{j}|y)p(\bar{i} \cap j|y)}\right) \tag{9}$$

Modified Yule's Q Measure (YQM) Hofmarcher et al. (2018)

$$J_{YQ}(i,j) = \ln\left(\frac{\left(p(i \cap j|y) + \frac{1}{2}\right)\left(p(\bar{i} \cap \bar{j}|y) + \frac{1}{2}\right) - \left(p(i \cap \bar{j}|y) + \frac{1}{2}\right)\left(p(\bar{i} \cap j|y) + \frac{1}{2}\right)}{\left(p(i \cap j|y) + \frac{1}{2}\right)\left(p(\bar{i} \cap \bar{j}|y) + \frac{1}{2}\right) + \left(p(i \cap \bar{j}|y) + \frac{1}{2}\right)\left(p(\bar{i} \cap j|y) + \frac{1}{2}\right) - \frac{1}{2}}\right) \tag{10}$$

A (B) indicates that event A (B) did not occur.

3 Usage of the BACE package

3.1 BACE main function

The BACE package defines two public functions: `BACE_GUI()` and `BACE_Print()`. The `BACE_GUI()` takes following parameters:

- **Dependent variable** as series [required].
- **Y lags** as integer with zero (only for time-series).
- **List of independent variables (X)** as list [may be null].
- **Constant in model** as integer [required]:
  1. Never – all models without constant,
  2. Always – all models with constant,
  3. Can be dropped – constant may be removed from or added to any model.
- **Model prior** as integer [required]:
  1. Binomial – binomial distribution,
  2. Beta-Binomial – beta-binomial distribution.
- **Prior average model size** as scalar with zero [required].
- **Significance level for the initial model** as scalar from $(0, 1)$ [required].
- **Number of the top ranked models** as integer [required].
- **Jointness analysis** as integer with zero [required][1]:
  1. None – we do not perform any jointness analysis,
  2. Ley-Steel Measure – we perform jointness analysis with Ley-Steel measure,
  3. Doppelhofer-Weeks Measure – we perform jointness analysis with Doppelhofer-Weeks measure.
- **Number of out-of-sample forecasts** as integer with zero [required].
- **Total number of replications** as integer [required].
- **Percentage of burn-in draws** as integer from (0, 99) [required].
- **Verbosity** as integer [required].
- **Show overall progress** as Boolean.

The `BACE_GUI()` function returns a bundle with all information needed to print results anytime. One can print the results by calling `BACE_Print(&RES)`, where `RES` is the bundle returned by the `BACE_GUI()` function.

[1] Values ± 999.000000 should be interpreted as $\pm\infty$.

3.2 The GUI way

Once you start **gretl**, you must open a data file and then you can load the relevant **BACE** package from the **gretl** server. In the main window, go to File → Function packages → On server heading. By selecting **BACE**, you will download the package on your local machine. Next, go to File → Function packages → On local machine and by selecting BACE from the list open a window similar to the one shown in Figure 1:

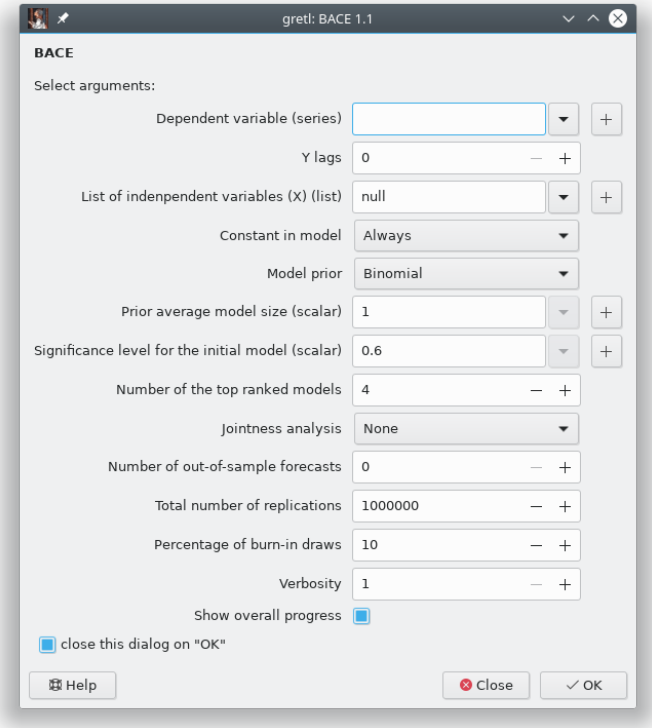


Figure 1: Main window for **BACE**.

According to Figure 1, we can specify the following entries in the GUI **BACE** window

- **Dependent variable** – The dependent variable.
- **Y lags** – number of lags for dependent variable (for time-series only).
- **List of independent variables** – Loading variables from the database, which must have been opened previously. Note that by default we assume that you want to estimate an intercept; therefore, a constant is implicitly included in the list of the variables.
- **Constant in model** – Do we include intercept? It can be: always, never or can be dropped.

- **Model prior** – Indicates the choice of model prior. One can employ the binomial model prior or the binomial-beta model prior. Note that the uniform model prior is a special case of the binomial model prior. Therefore, in fact, our package allows for three types of priors.

- **Prior average model size** – Specifies the prior expected model size $\mathsf{E}(\Xi)$. The expected model size may range from 0 to K. The default value equals 1. It is the smallest reasonable prior expected model size. It means that we penalize large models and we assign high probability to small ones following Occam's razor principle.

  Note that for the binomial model prior and with $\mathsf{E}(\Xi) = 0.5K$ one can define the uniform prior on the model space. For example, for $K = 10$ regressors we can define the uniform prior with $\mathsf{E}(\Xi) = 5$. For $\mathsf{E}(\Xi) < 5$ we assign high probability to small models. The smaller the prior expected model size $\mathsf{E}(\Xi)$, the less probable are larger models.

- **Significance level for the initial model** – Defines the significance level which is used to build the initial model. An explanatory variable enters the initial model if its p value is less than the significance level. If the significance level equals 1, the initial model will be randomly chosen (with equal probability) from all available models. Note that if all available explanatory variables enter the initial model, you will get the following **gretl** error message "No independent variables were omitted".

- **Number of top ranked models** – Specifies the number of best models for which detailed information is stored.

- **Jointness analysis** – If `None` (the default), the jointness analysis is omitted. Alternatively, one can choose the jointness measures of Ley and Steel (2007) or Doppelhofer and Weeks (2009).

- **Number of out-of-sample forecasts** – Defines the total number of out-of-sample forecasts of the dependent variable.

- **Total number of replications** – Defines the total number of iteration draws to be sampled.

- **Percentage of burn-in draws** – Specifies the number of burn-in replications, calculated as the percentage of the total number of iteration draws.

- **Verbosity** – An integer value of 1 or 2; the default is 1, which allows to see the basic Bayesian Averaging of Classical Estimates results. If Verbosity equals 2, a more detailed description of the analysis is provided (initial model, speed of convergence, estimation results for top ranked models).

- **Show overall progress** – The output is flushed every 10% of MC^3 and the progress is printed. Note: this consumes a lot of CPU time.

3.3 The script way

3.3.1 Script

The **BACE** package can also be used inside `Hansl` scripts. A very minimal code could be as follows:

```
open greene9_1.gdt
include BACE.gfn

list greene = capital labor nfirm

BACE(valadd, greene)
```

The above code triggers BACE analysis with default option described on page **??**. On contrary, the corresponding "big call", i.e. with setting all possible parameters, would be as follows:

```
bundle opts = defbundle("model_prior", 2, "avg_model_size", 1.5, \
 "alpha", 0.6, "Nrank", 4, "do_joint", 0, "Npredict", 0, \
 "Nrep", 1000, "burn", 10, "verbosity", 2, "show_progress", 1, \
 "do_plot", 1, "model_type", "linear", "const_type", 1, \
 "bic_type", 1, "data_trans", 0)

BACE(valadd, greene, opts)
```

or alternatively with the new syntax for creating a bundle on-the-fly:

```
BACE(valadd, greene, _(model_prior=2, avg_model_size=1.5, \
 alpha=0.6, Nrank=4, do_joint=0, Npredict=0, Nrep=1000, \
 burn=10, verbosity=2, show_progress=1, do_plot=1, \
 model_type="linear", const_type=1, bic_type=1, data_trans=0))
```

Since some parameters in option bundle are over-loadable, the equivalent call looks like this:

```
BACE(valadd, greene, _(model_prior="beta-binomial", \
 avg_model_size=1.5, alpha=0.6, Nrank=4, do_joint=0, Npredict=0, \
 Nrep=1000, burn=10, verbosity=2, show_progress=1, do_plot=1, \
 model_type="linear", const_type="always", bic_type="bic", \
 data_trans="none"))
```

The above example consists of three blocks.

```
open greene9_1.gdt
include BACE.gfn
set seed 1000000

list greene = capital labor nfirm

scalar y_lags = 0
scalar with_const = 1
scalar model_prior = 1
scalar prior_avg_model_size = 1.5
```

```
scalar alpha = 0.6
scalar top_ranked_model = 4
scalar joint = 1
scalar forecasts = 0
scalar Nrep = 2000
scalar Nburn = 10
scalar verbosity = 1
scalar progress = 1

BACE_GUI(valadd, y_lags, greene, with_const, model_prior, prior_avg_model_size,
    alpha, top_ranked_model, joint, forecasts, Nrep, Nburn, verbosity, progress)
```

The above example consists of three blocks. The first block is just opening of the so called `greene9_1` dataset, which is bundled in every standard gretl installation. This dataset contains cross-sectional data on manufacturing of transportation equipment presented as Table 9.1 in Greene (1999)[2]. We also set the seed in order to exact reproduction of the results.

The second block is the definition of the `greene` list which contains three independent variables available in the `greene9_1` dataset.

The third block contains the definition for Bayesian Averaging of Classical Estimates analysis: no lags for Y variable (not applicable since we have cross-sectional data), constant always included, binomial model prior, prior average model size set to 1.5 (which means that we have uniform model prior), significance level for the initial model set to 0.6, 4 top ranked models, jointness analysis with Ley-Steel Measure, without out-of-sample predictions, 10000 replications with 10% burn-in draws, basic output (verbosity set to 1) and progress overall progress is printed.

Suppose we want to run the same analysis but following Occam's razor principle. The simplest way is setting the prior average model size to 1. We also want to have "random" constant, i.e. intercept can be dropped in any model. We also want to save results of the BACE analysis in the bundle `results_b`, the code should be as follows:

```
prior_avg_model_size = 1
with_const = 2

results_b = BACE_GUI(valadd, y_lags, greene, with_const, model_prior,
    prior_avg_model_size, alpha, top_ranked_model, joint, forecasts,
    Nrep, Nburn, verbosity, progress)
```

Suppose we want to calculate Doppelhofer-Weeks Measure instead of already computed Ley-Steel Measure. Since the type of jointness measure is controlled by `do_join` field in the result bundle, the only thing we have to do is to set the value of `do_join` to 2 and print BACE results again. The code should be as follows:

[2]This dataset is also available at link http://people.stern.nyu.edu/wgreene/Text/Edition7/TableF7-2.txt.

```
results_b.do_joint = 2
BACE_Print(&results_b)
```

3.4 Output

If you select the appropriate entries in the GUI **BACE** window, our package returns: Posterior Inclusion Probabilities (PIP), the averages of the posterior means and standard deviations of parameters ("Mean" and "Std.Dev.", respectively) and the posterior mean and standard deviation of each coefficient conditional on the variable being included in the model ("Cond.Mean" and "Cond.Std.Dev."). Furthermore, the package also returns the predictive results for the dependent variable ("Mean" and "Std.Dev.").

Let us consider the data used in previous example (`australia.gdt`). This dataset contains the Australian macro data used in Johansen (1995) and originally supplied by Tony Hall. The variables used in the text are:

- lpau – log of Australian CPI (dependent variable),
- lpus – log of US CPI,
- le – log of exchange rate (price of US dollars in Aus. dollars),
- iau2 – 5-year Treasury bond rate, Australia,
- ius2 – 5-year Treasury bond rate, USA.

Suppose we want to perform BACE analysis up to $ADL(2,2)$ specification, i.e. all possible combinations from $ADL(0,0)$ to $ADL(2,2)$. The example script would be as follows:

```
set verbose off
include BACE.gfn
open australia.gdt
genr time

# We create lagged values
adl_lags = 2
lags(adl_lags, ius2)
lags(adl_lags, iau2)
lags(adl_lags, e2)
lags(adl_lags, lpus)
lags(adl_lags, le)

# We define list with all X-es (including lags)
list X_list = time ius2 ius2_* iau2 iau2_* e2 e2_* lpus lpus_* le le_*

set seed 1000000
scalar y_lags = 2
```

```
scalar with_const = 2
scalar model_prior = 1
scalar prior_avg_model_size = (nelem(X_list) + y_lags + (with_const > 0)) / 2
scalar alpha = 0.6
scalar top_ranked_model = 4
scalar joint = 0
scalar forecasts = 4
scalar Nrep = 20000
scalar Nburn = 10
scalar verbosity = 2
scalar progress = 0

RES = BACE_GUI(lpau, y_lags, X_list, with_const, model_prior, prior_avg_model_size,
    alpha, top_ranked_model, joint, forecasts, Nrep, Nburn, verbosity, progress)
```

In the above analysis all variables (including dependent variable $lpau_t$) are lagged by 2 periods. Additionally we compute 4 forecasts:

```
Posterior moments (unconditional and conditional on inclusion):
                PIP         Mean     Std.Dev.    Cond.Mean Cond.Std.Dev
lpau_1     1.000000     0.894595     0.091656     0.894595     0.091656
  ius2     0.996500    -0.418560     0.102196    -0.420030     0.099314
lpus_2     0.790611    -0.303217     0.212152    -0.383523     0.161647
lpus_1     0.780500     0.356760     0.260342     0.457092     0.202432
  iau2     0.752222     0.224487     0.161678     0.298432     0.112617
  le_2     0.506944    -0.027670     0.046149    -0.054582     0.052271
 const     0.392778    -0.074241     0.125663    -0.189015     0.136050
  e2_2     0.335667     0.002317     0.033496     0.006903     0.057540
  lpus     0.296222     0.037894     0.107134     0.127924     0.165014
  le_1     0.282500    -0.023085     0.062683    -0.081716     0.095485
lpau_2     0.253056     0.037914     0.087083     0.149825     0.114894
  e2_1     0.245222     0.015275     0.052532     0.062292     0.091240
  time     0.233778     0.000053     0.000269     0.000226     0.000520
iau2_1     0.231389     0.035210     0.090726     0.152166     0.133328
    le     0.211667     0.010063     0.042201     0.047542     0.081437
    e2     0.176222    -0.005057     0.034623    -0.028698     0.078258
ius2_1     0.175222     0.021296     0.072659     0.121539     0.133961
ius2_2     0.140111    -0.009023     0.048048    -0.064400     0.113626
iau2_2     0.132000     0.005342     0.042150     0.040469     0.109717

---------------------------------

Posterior probability of models:
Model 1:        0.024278
Model 2:        0.020556
```

```
Model 3:        0.012889
Model 4:        0.009278
Total probability of the models in ranking (numerical): 0.067000

Correlation coefficient between the analytical
and numerical probabilities of the above models: 0.932557

---------------------------------

Predictive results:
                  lpau          Mean       Std.Dev.
1990:2        5.334649       5.340552       0.007950
1990:3        5.341856       5.356902       0.008400
1990:4        5.368310       5.378660       0.010094
1991:1        5.366443       5.395817       0.013705
```

References

Albis MLF, Mapa DS. 2014. Bayesian Averaging of Classical Estimates in Asymmetric Vector Autoregressive (AVAR) Models. MPRA Paper 55902, University Library of Munich, Germany.
URL https://ideas.repec.org/p/pra/mprapa/55902.html

Atwood CL. 1980. Estimators for the truncated beta-binomial distribution .
URL https://www.osti.gov/biblio/5359803

Białowolski P, Kuszewski T, Witkowski B. 2014. Bayesian averaging of classical estimates in forecasting macroeconomic indicators with application of business survey data. *Empirica* **41**: 53–68.
URL https://ideas.repec.org/a/kap/empiri/v41y2014i1p53-68.html

Doppelhofer G, Weeks M. 2009. Jointness of Growth Determinants. *Journal of Applied Econometrics* **24**: 209–244.

Greene W. 1999. *Econometric Analysis*. Upper Saddle River: Prentice Hall, 4th edition.

Hofmarcher P, Crespo Cuaresma J, Grün B, Humer S, Moser M. 2018. Bivariate jointness measures in Bayesian Model Averaging: Solving the conundrum. *Journal of Macroeconomics* **57**: 150–165. ISSN 0164-0704.
URL https://www.sciencedirect.com/science/article/pii/S0164070418300314

Johansen S. 1995. *Likelihood-Based Inference in Cointegrated Vector Autoregressive Models*. Oxford University Press.

Kass RE, Raftery AE. 1995. Bayes Factors. *Journal of the American Statistical Association* **90**: 773–795. ISSN 0162-1459.

Leamer E. 1978. *Specification Searches*. New Jersey, USA: John Wiley & Sons. ISBN 0471015202.

Ley E, Steel M. 2007. Jointness in Bayesian Variable Selection with Applications to Growth Regression. *Journal of Macroeconomics* **29**: 476–493.

Mitchell TJ, Beauchamp JJ. 1988. Bayesian Variable Selection in Linear Regression. *Journal of the American Statistical Association* **83**: 1023–1032.
URL https://www.tandfonline.com/doi/abs/10.1080/01621459.1988.10478694

Strachan RW. 2009. Comment on 'Jointness of growth determinants' by Gernot Doppelhofer and Melvyn Weeks. *Journal of Applied Econometrics* **24**: 245–247.
URL https://onlinelibrary.wiley.com/doi/abs/10.1002/jae.1050

Tripathi RC, Gupta RC, Gurland J. 1994. Estimation of parameters in the beta binomial model. *Annals of the Institute of Statistical Mathematics* **46**: 317–331. ISSN 0020-3157.
URL http://link.springer.com/10.1007/BF01720588

van Dijk D. 2004. Forecasting US Inflation Using Model Averaging. Econometric Society 2004 Australasian Meetings 143, Econometric Society.
URL https://ideas.repec.org/p/ecm/ausm04/143.html

Xavier Sala-i-Martin and Gernot Doppelhofer and Ronald I Miller. 2004. Determinants of Long-Term Growth: A Bayesian Averaging of Classical Estimates (BACE) Approach. *American Economic Review* **94**: 813–835.
URL https://ideas.repec.org/a/aea/aecrev/v94y2004i4p813-835.html